Direct closure induction for iPRA

Removing the existential witness from induction, with exact proof translations and complete checkers

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

The current ConciseIPRA presentation packages literal-equation induction inside an existential introduction rule. A simpler primitive interface introduces the conditional universal closure directly:

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash \mathsf{Q}_y E(y)}, \qquad \mathsf{Q}_y E(y) := E(0) \to \forall y\, E(y).$$

The existential binder retains only its case-introduction rule. The replacement removes one numerical witness and one bound coordinate from the induction schema. It changes neither terms, formulas, contexts, nor the consequence relation. Effective translations in both directions preserve the final sequent and have linear inference-node bounds. The whole-domain bi-interpretation with full first-order intuitionistic PRA therefore transfers unchanged.

The direct closure rule itself is already a derived theorem in the repository. The contribution here is its selection as primitive, explicit bidirectional proof compilers, and a controlled implementation comparison. Two independently executable strict Python checkers, differing only in the progression branch, measure 5,199 versus 5,116 normalized bytes, 1,187 versus 1,169 significant tokens, and 1,358 versus 1,337 AST nodes. The smaller source has a larger gzip representation. These checkers do not improve the repository's 529-byte compact frontier. No globally simplest axiomatization or new arithmetic foundation is claimed.

Result and limitation. There is an exact simplification of the primitive induction interface and a measurable reduction in matched, explicit proof-checker implementations. A presentation below the current best compact-checker frontier has *not* been established. This is a modest next iteration, not the requested substantial size breakthrough.

Contents

1 Comparison point and exact scope **3**

2 The complete proposed axiomatization **3**

3 Structural properties and exact equivalence **5**

4 Induction is now an immediate composition **6**

5 Recovering full intuitionistic logic **6**

6 Bi-interpretability and equiconsistency **7**

7 Complete checker implementations and measurement **9**

8 Executable proof translations and validation **10**

9 Lean migration and its present verification boundary **11**

10 A small certificate tradeoff and a rejected false improvement **11**

11 Conclusion and next concrete step **12**

A Attribution for the complete inherited bridge **13**

B Complementary introductions with one local assumption **13**

C Arithmetic strength and both ordinary interpretation composites **18**

D Internal scalar predecessor bootstrap **20**

E From literal-equation induction to full intuitionistic PRA **24**

F Pairing, primitive recursion, and internal numerical composites **29**

G Standalone direct-closure checker **32**

1 Comparison point and exact scope

The source comparison is pinned to *ProveIt*, commit

`b17dd5487f34adea473a0204c5320fb3eeb8910e`,

under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. This was the default-branch head retrieved at the beginning of this investigation. Later branch movement does not alter the pinned comparison. The source snapshot lists exact Git blob identifiers and SHA-256 hashes. The repository's README identifies the native scalar core and its smallest complete implementations; the formal files contain the ordinary bi-interpretation and native formula round trip [1, 3, 4].

The task is about full first-order iPRA, not merely its equational fragment or its standard numerical functions. We use the usual intuitionistic first-order equality logic with one function symbol for each fixed primitive-recursive description, its defining equations, the successor axioms, and quantifier-free induction with parameters and retained assumptions. The two interpretable theories must reconstruct all formulas and their proof rules. Merely exhibiting all primitive-recursive functions on the standard natural numbers would not suffice.

The simplification developed here does not exploit a change of numerical domain. Both numerical interpretation domains are whole, equality is actual equality, and the internally definable isomorphism for each numerical composite is the graph $x = y$. This is stronger than the equiconsistency requirement by itself.

Attribution. The scalar iterator, the Q/D syntax, the logical reconstruction, and the arithmetic bridge are inherited. In particular, `NativeEq.lean` already proves `NPrf.qIProg`, the exact rule we choose as primitive [2]. It would be misleading to call this a newly discovered induction principle. The new presentation changes which admissible rule is fundamental and supplies the exact compiler and measurement evidence for that choice.

2 The complete proposed axiomatization

Call the presentation $\mathcal{C}_Q$. Its entire primitive inventory follows.

2.1 Terms and formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u in the update body h is bound; it does not bind occurrences in the counter n or initializer a. Every update is fixed finite syntax with passive numerical parameters. No object-language universal evaluator, quantification over function codes, or higher-order domain is postulated.

Native formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The variables x, y in D are distinct. The designated contradiction is $\bot_* := (0 = S0)$. Contexts Γ are finite reusable lists of formulas. All substitutions below avoid capture; bound and eigenvariables may first be renamed.

The intended ordinary readings are

$$(s = t)^\circ = (s = t), \qquad (\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$
$$(\mathsf{D}_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y\, B^\circ(x, y)).$$

These readings explain the symbols; they are not additional native axiom schemas.

2.2 The two computation schemas

In every retained context:

$$\frac{}{\Gamma \vdash I(0, a; u.h) = a}\ \text{I0}, \qquad \frac{}{\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]}\ \text{IS}.$$

2.3 All six other primitive inference schemas

$$\frac{}{\Gamma \vdash A}\ \text{Hyp} \quad (A \in \Gamma),$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}\ \text{DE},$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}\ \text{DCase},$$

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash \mathsf{Q}_y E(y)}\ \text{QInd} \quad (E \text{ a literal equation}).$$

In Tr, $F(v)$ is any native formula template, including templates with occurrences under binders. In DE, b is fresh for the retained context, the conclusion, and all fixed parameters of the matrix. In DCase, z is fresh for the retained context, the witness t, and every fixed matrix parameter. In QInd, z is fresh for the retained context and every fixed parameter of E. Displayed local assumptions are discharged.

The phrase *literal equation* means an equation at the matrix root. An equation beneath a Q or D binder is not a literal equation. That restriction is part of the theory and part of the executable checkers.

This is the whole primitive presentation: eight schemas when computation leaves and Hyp are counted. There is no primitive reflexivity, successor injectivity, predecessor, pairing, equality test, numerical coverage, ordinary connective rule, or additional induction schema. Weakening, exchange, contraction, renaming, numerical substitution and cut are admissible properties, not unlisted certificate instructions.

2.4 What has been removed

The old calculus $\mathcal{C}_D$ has the same first seven schemas but replaces QInd by

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)}\ \text{DProg}.$$

It has two variables in its matrix and an explicit existential witness t. QInd has one matrix variable and no witness. DCase now gives the only primitive introduction of D; QInd gives the primitive introduction of Q. The number of schemas is unchanged, but induction no longer performs existential bookkeeping.

3 Structural properties and exact equivalence

Lemma 3.1 (Structural admissibility)**.** *In $\mathcal{C}_Q$, weakening, exchange, contraction, capture-free renaming, numerical substitution, and ordinary-assumption cut are admissible.*

Proof. Induct on derivations. For weakening and numerical substitution, rename local eigenparameters away from the finitely many retained expressions being inserted. Every computation schema is stable under substitution. The major and minor formulas in Tr and QE substitute coherently. DE and DCase retain their respective fresh variables after renaming. For QInd, substitution preserves the equation at the matrix root and acts only on its passive parameters; its distinguished fresh variable continues to be fresh. Thus the same QInd rule applies to the substituted premise. Exchange and contraction change only reusable context membership.

For cut, replace each use of the discharged assumption by its given proof, weakened into the local context, again renaming eigenparameters first. This proof substitution preserves every rule side condition. In particular, no test on the literal spelling of a numerical witness changes under substitution: the new induction rule has no such witness. □

Theorem 3.2 (Identity-language equivalence)**.** *For every finite native context Γ and native formula A,*

$$\Gamma \vdash_{\mathcal{C}_Q} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_D} A.$$

The transformations are effective on finite derivations and leave the final sequent literally unchanged.

Proof. All common rules translate recursively without modification. There is exactly one changed case in each direction.

From $\mathcal{C}_Q$ to $\mathcal{C}_D$. Suppose the translated premise of QInd is $\pi : \Gamma, E(z) \vdash E(Sz)$. Choose a fresh variable u absent from E and the retained context. Old DProg, with witness zero and unused bound coordinate u, gives

$$\Gamma \vdash_{\mathcal{C}_D} \mathsf{D}_{u,y} E(y).$$

Eliminate this formula by DE. Its local hypothesis is precisely $\mathsf{Q}_y E(y)$, since the existential coordinate does not occur. Return that hypothesis by Hyp and close DE. This proves the required closure under exactly the original retained context. This is the construction already formalized as `NPrf.qIProg`.

From $\mathcal{C}_D$ to $\mathcal{C}_Q$. Suppose an old DProg has matrix $E(x, y)$, witness t, and translated premise $\pi : \Gamma, E(t, z) \vdash E(t, Sz)$. Set $F(y) := E(t, y)$. The witness t is now a passive parameter of this fixed literal-equation template. QInd gives

$$q : \Gamma \vdash_{\mathcal{C}_Q} \mathsf{Q}_y F(y).$$

Use DCase for the original matrix E and the original witness t. Its local assumption is $F(0)$. Weaken q into that context and use QE at Sz to derive $F(Sz)$. DCase concludes the original $\mathsf{D}_{x,y} E(x, y)$. The old freshness condition on t is exactly the one needed to make it a passive parameter in QInd. The original premise π is used once. Structural induction on the original proof now gives both translations. □

Proposition 3.3 (Exact inference-node bounds)**.** *If a $\mathcal{C}_Q$ derivation has N inference nodes and r QInd nodes, the displayed translation has $N + 2r \leq 3N$ old-calculus nodes. If a $\mathcal{C}_D$ derivation has N nodes and r DProg nodes, its translation has $N + 3r \leq 4N$ new-calculus nodes.*

Proof. QInd is replaced by DProg, DE, and Hyp, adding two nodes. DProg is replaced by QInd, DCase, QE, and Hyp, adding three. In both replacements the transformed premise occurs once. All other nodes are copied once. □

The bounds concern inference trees. They are not bounds on formula length, serialized JSON bytes, runtime, or normalized checker source. The executable translators record these exact equalities on their regression examples.

4 Induction is now an immediate composition

Corollary 4.1 (Contextual equation induction)**.** *For every literal equation E,*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}$$

is admissible, with the usual freshness condition.

Proof. Apply QInd to the step premise and QE to the resulting closure and the given base proof. □

For a concrete example, let $J(n, a) = I(n, a; u.u)$. We have $J(0, a) = a$ by I0 and $J(Sz, a) = J(z, a)$ by IS. Under the induction assumption $J(z, a) = a$, transport yields $J(Sz, a) = a$. Thus QInd proves

$$\mathsf{Q}_y(J(y, a) = a),$$

and QE with the base computation gives $J(t, a) = a$ for any t. The package contains closed and parameterized certificates for this example and verifies their translations both ways.

Allowing QInd at arbitrary formulas would be an actual strengthening: the same two-rule composition would supply induction for arbitrary quantified formulas. The literal-equation guard cannot be omitted merely because it makes a checker shorter. Conversely, forcing progress for all atomic D introductions would lose the atomic case rule; it is not the present construction. Here that case rule is retained unchanged.

5 Recovering full intuitionistic logic

Theorem 3.2 transfers the entire existing logical reconstruction, not just its closed theorems. Some details are given here to identify precisely what is inherited; the complete written construction is reproduced in Appendix B.

Equality reflexivity follows from computation and transport. Put $r = I(0, t; u.0)$ and use $r = t$ twice in Tr with motive $v = t$; the conclusion is $t = t$. Symmetry, transitivity and congruence follow by the usual explicit equality motives.

Numerical cases for every native formula follow without induction: from $\Gamma \vdash F(0)$ and $\Gamma \vdash F(Sz)$, DCase with an unused existential coordinate produces $\mathsf{D}_{u,y}F(y)$. DE supplies $\mathsf{Q}_y F(y)$ and QE gives $F(t)$. This is a zero/successor case rule, not an induction rule for arbitrary F.

Let $C(v,a,b) := I(v,a;u.b)$. Its equations distinguish zero from a successor. Recursive gates on fixed formula syntax give G_vA with

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A.$$

A second recursive construction gives a selector $M_v(A,B)$ with $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$. Unequal formula depths are aligned by the derivable padding $A \dashv\vdash \mathsf{D}_{u,y}G_yA$. These are finite syntactic macros, not quantification over formulas in the object theory.

With $p(z) = I(z,0;u.C(u,1,0))$, the full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = S0). \end{aligned}$$

All displayed unused variables are chosen fresh. The gate and selector endpoints, cases and DE/QE establish the ordinary intuitionistic introduction and elimination rules. In particular, the two fixed instances $p(1) = 1$ and $p(2) = 0$ give both conjunction projections. Disjunction elimination uses numerical cases on its existential selector.

Under $0 = S0$, computation and transport equate arbitrary numerical terms using $C(0,s,t) = s$ and $C(1,s,t) = t$. External induction on finite formula syntax then derives every native formula. Thus the designated contradiction has the required explosion rule without postulating it separately.

The auxiliary Q_yB is interderivable with $\mathsf{D}_{u,y}B$ for unused u. One direction uses DE and Hyp. The other uses DCase and QE under the local base assumption. This works for arbitrary matrices; it does not license unrestricted QInd.

6 Bi-interpretability and equiconsistency

6.1 The ordinary realization

Let T_{eq} be ordinary intuitionistic first-order equality logic over the same staged iterator terms, with I0 and IS, contextual induction for literal equations, numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

and the explicit falsity bridge $(0 = S0) \to \bot$. Staging means that each fixed update description is introduced only after its component descriptions; no numerical input is interpreted as an arbitrary program to evaluate.

Every $\mathcal{C}_Q$ rule is sound under the displayed ordinary readings. For QInd, temporarily assume $E(0)$, apply contextual equation induction to the supplied step, introduce the universal quantifier, and discharge the base. DCase uses numerical coverage under the retained base assumption. All eliminations and transport are ordinary logical inferences.

Conversely, the recovered logical operations give ordinary equality logic; the new QInd/QE composition gives equation induction; the derived cases give coverage; and native explosion supplies the falsity bridge under the interpretation. The bridge must remain explicit in the ordinary theory: positive equations alone admit a singleton model.

6.2 The numerical compiler and its order of dependencies

The inherited arithmetic proof proceeds in an order that avoids assuming the very functions being constructed:

1. Scalar iteration, equation induction and cases yield a fixed term P with $P0 = 0$ and $P(Sn) = n$.
2. Truncated subtraction and a numerical characteristic of actual equality are constructed and proved correct on the entire domain.
3. Each fixed quantifier-free formula is converted to a characteristic equation. Equation induction therefore gives its contextual quantifier-free induction instance.
4. Pairing and projections are then constructed, and every fixed general primitive-recursive description is compiled into a scalar iterator term.

The complete predecessor, reflection and primitive-recursion arguments are supplied in the attributed appendices.

For example, with pairing π and projections L, H, a recursion $f(0, \vec{p}) = g(\vec{p})$ and $f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$ is implemented by

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\,, \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$, and the projection identities prove the required defining equations. This construction is external recursion on a finite function description.

Theorem 6.1 (Whole-domain bi-interpretation)**.** *The ordinary realization of $\mathcal{C}_Q$ is bi-interpretable with full first-order* iPRA*. Both numerical composites are internally isomorphic to the identity by the graph $x = y$. All formula composites are provably equivalent, and $\mathcal{C}_Q$ and* iPRA *are equiconsistent.*

Proof. The pinned development establishes the native $\mathcal{C}_D/T_{\mathrm{eq}}$ formula round trip and the T_{eq}/iPRA numerical and formula round trips. Theorem 3.2 preserves every native sequent on the same syntax, so these results already transfer.

More explicitly, iPRA interprets a fixed initialized iterator by primitive recursion whose update ignores the recursion counter. It proves the iterator equations, coverage, separation and equation induction. The reverse interpretation uses the scalar compiler and the preceding predecessor and equality-reflection results. Internal equation induction proves uniqueness of each fixed iterator and each fixed primitive recursion with its given defining equations. External induction on descriptions therefore proves that double translation is internally equal to the original function at every argument, including nonstandard arguments in models.

The numerical domain and equality do not change. For logical syntax, the recovered macros have their stated ordinary meanings, and in the native calculus

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big).$$

Forward, DE opens its witness; under the base the local closure supplies every instance, so introduce universal, implication and existential macros. Backward, eliminate the existential and use DCase: the retained base, implication and universal give the successor premise. The analogous Q identity follows by the unused-witness conversion. External induction on formulas proves both formula composites. The explicit falsity bridge preserves contradiction. Effective proof translations in both directions give equiconsistency; there is no claim of a theory proving its own consistency. □

The baseline files `BiInterpretation.lean` and `NativeComposite.lean` contain the formal statements `biInterp`, `teq_consistent_iff`, `NEquiv.ofFm_toFm`, and `NPrf.iff_teq`. This report inspected their source. It did not rebuild the repository's Lean project, and it does not equate the Python regression evidence with proof-assistant verification.

7 Complete checker implementations and measurement

7.1 A controlled, straightforward comparison

The two strict reference programs are standalone modules with the same parser, signature, scope validator, capture-free substitution, five common inference branches, two computation branches and public JSON interface. They differ only in their progression branch and identifying docstring. The old program explicitly implements witnessed DProg; the new program explicitly implements QInd. There is no imported logical kernel, external axiom table, uncharged compiler or certificate generator in either program.

Complete program	Bytes	Tokens	AST	Gzip 9
Matched explicit old rules	5,199	1,187	1,358	1,303
Direct-Q rules	**5,116**	**1,169**	**1,337**	1,322
Difference (new minus old)	−83	−18	−21	+19

The byte reduction is approximately 1.60%. Raw source lengths are 5,731 and 5,645 bytes; raw gzip sizes are 1,498 and 1,518 bytes. The gzip coordinates are informational, as requested, and were not used as an optimization target. A shorter normalized source need not compress better.

The source comparison directly isolates the structural change: remove the induction witness, its scope check, and the operation substituting it into a two-variable matrix. Both files are produced by a builder with one common body and two progression fragments, and both remain executable without the builder. This is a controlled style comparison, not evidence that either reference program is globally source-minimal.

7.2 The existing compact frontier is still smaller

Pinned repository profile	Bytes	Tokens	AST	Gzip 9
Neutral scope: byte profile	529	266	310	333
Prevalidated elimination: token profile	576	261	292	345
Coverage and kernel: AST profile	613	263	289	349

The coordinate minima are attained by different programs. None of the new programs improves these minima. The compact implementations combine formula and numerical traversals, alias decoding, local fresh-variable scopes, shared continuation hypotheses and overloaded certificate fields. The strict reference interface instead validates a tagged, well-sorted, well-scoped syntax explicitly. The semantics on canonical native formulas coincide, but their certificate languages are different.

The old compact checker also recognizes the new theory after composing with the equivalence theorem. That fact is useful for proof production, but does not show that making QInd primitive caused a 529-byte implementation. We do not assign the compact byte count to the direct reference implementation or present the 83-byte matched reduction as an improvement below the historical frontier.

7.3 Measurement protocol

The runtime is CPython 3.12.14 with zlib 1.3.2. Genuine module, class and function docstrings are removed; the source is normalized by `ast.unparse` followed by one final newline. Significant tokens exclude comments, indentation, line breaks and end markers. AST counts use `ast.walk` on the normalized module. The entire module is charged, including imports, syntax and scope checks, JSON input, and the verdict. The common Python runtime and standard library are excluded under the existing repository convention.

Gzip uses `compresslevel=9,mtime=0`, including the gzip wrapper. Both raw and normalized compressed sources are included. The machine receipt records hashes, exact versions and every measured file. Different Python or zlib versions may change implementation-specific measurements. No abstract minimality theorem follows from a finite source comparison.

8 Executable proof translations and validation

8.1 Strict certificate syntax

Expressions are tagged JSON arrays. Variables are `["v",i]`, zero is `["z"]`, successor is `["s",t]`, and iteration is `["i",n,a,h]`. The update binds variable zero. Equations, closures and paired formulas use tags `"e"`, `"q"` and `"d"`; their field counts and binding arities are validated. The D matrix has y at index zero and x at index one.

A certificate states its number of free numerical parameters, retained context, goal, and proof tree. Every declared input formula is checked before derivation. Each rule validates its local templates; the derived formula is then compared with the declared goal. Integers used as indices must be actual nonnegative Python integers, so JSON booleans do not silently serve as variable or hypothesis indices. There is no evaluation of certificate text as Python.

Theorem 8.1 (Strict checker correctness)**.** *Under adequate resources, the direct-*Q *checker accepts a certificate exactly when its stated well-scoped proof tree is a* $\mathcal{C}_Q$ *derivation of its stated sequent. Every finite* $\mathcal{C}_Q$ *derivation has such a certificate. The analogous statements hold for the matched old checker and* $\mathcal{C}_D$*.*

Proof. The signature validator is a structural recursion on finite expressions. Its variable case checks the current numerical scope; every other case checks the exact tag, sort and number of children, increasing scope only in the prescribed binding fields. Simultaneous substitution extends an image map under b binders by fixing the first b indices and shifting each substituted outer term by b. Structural induction gives the usual capture-free substitution and weakening lemmas.

Now induct on a successful derivation call. Hyp returns a valid context entry. The computation cases construct precisely I0 and IS. Tr checks its equality major and the source instance before returning the target instance. QE checks its closure and base. DE checks its fresh-witness branch against the shifted conclusion. DCase checks its witness-instantiated matrix under the shifted zero instance and at the successor of the new variable. QInd checks a literal equation and exactly its progression premise. In each case the returned formula is the displayed rule's conclusion under its stated side conditions. Unrecognized rules fail. Initial well-scoping and these local checks establish soundness.

Conversely, encode a finite derivation using de Bruijn indices, renaming eigenvariables first. Every valid rule application satisfies the corresponding branch tests. Induction on the derivation proves acceptance. No normalization of arbitrary raw aliases is required in this strict interface. □

The compilers in `code/translate.py` are untrusted producers. Their outputs are accepted only after the independent destination checker checks them. The compiler implements numerical substitution and context embedding for proof trees, including local binders and retained-hypothesis indices. These operations realize the two constructions in Theorem 3.2.

The principal suite tests computation, reflexivity, literal equation induction, DCase, actual use of the fresh closure in DE, transport under binders, free parameters, both proof translations and their round trips. It also rejects wrong conclusions, unscoped terms, Boolean indices, negative hypothesis indices, escaped eigenvariables and compound progression. Tests run the public command-line checkers in ordinary Python and with `-O`; the checks do not depend on assertions. Additional targeted review tests exercise actual retained-hypothesis reindexing and nonvacuous parameterized witnesses. Exact counts and review notes are distributed in `evidence/`.

Finite tests are implementation evidence. The universal equivalence and interpretation claims are the written mathematical theorems above and their attributed inherited dependencies.

9 Lean migration and its present verification boundary

The proposed migration replaces the native constructor `dIProg` by `qIInd`. The matrix terms move from `Tm(n+2)` to `Tm(n+1)`, and the witness argument `t:Tmn` disappears. The local constructor shrinks from 270 to 214 source bytes excluding its documentation; its explicit term inputs fall from three to two, and two witness-instantiation expressions disappear. These are local measurements, not a total Lean-project size comparison.

The included patch also updates the corresponding induction cases in weakening, substitution, cut and native soundness. It restores `NPrf.dIProg` as a derived compatibility theorem, while `NPrf.qIProg` becomes a one-line alias for the new constructor. Keeping the old derived interface should reduce disruption for downstream proofs.

Verification status: the patch passed `gitapply--check` against the pinned source copies. It has not been compiled or checked by Lean in this environment. Patch applicability is not elaboration or kernel verification. The patch is a reviewable migration proposal, not a claim that the formal development has already been updated successfully.

10 A small certificate tradeoff and a rejected false improvement

The ancillary `experiments/counter_vectors/` directory explores an independent change to the inherited compact checker. It supplies the computation counter as an empty or nonempty source vector. The empty vector selects zero computation; a vector beginning with n selects successor computation, even when n is the literal zero term. Consequently $I(S0, a; u.h)$ can be certified directly, without first replacing its predecessor by a syntactically nonzero representative of zero.

The sound canonical branch uses `s[:1]or0`. The vector's head is used in the recursive call; additional tail annotations do not change the computed predecessor. The whole checker has 533 normalized bytes, 271 tokens, 314 AST nodes, and a 336-byte gzip representation. It costs four bytes more than the 529-byte baseline. Its 300 process checks have zero mismatches. It is a certificate simplification with a size cost, not the selected answer to the size-minimization problem.

A tempting 528-byte candidate omits `or0`. It then writes the empty array in base counters where the established logical and substitution protocol uses scalar zero. Both expressions have the same intended numerical decoding, but the checker compares raw syntax. A canonical base certificate fails, and no universal alias-conversion proof was established. Therefore the 528-byte source is included only

as a rejected experiment and is not counted as a complete proof checker. This is a completeness gap, not a demonstrated proof of a false theorem.

11 Conclusion and next concrete step

Choosing direct closure induction removes an unnecessary existential witness from the primitive induction interface. The presentation is fully specified, preserves every native sequent, and retains the whole-domain bi-interpretation and equiconsistency with iPRA. The change makes the corresponding explicit checker branch shorter, and the executable proof compilers make the equivalence reviewable at the certificate level.

The outstanding target is a genuinely smaller complete implementation below the current compact frontier. This investigation has not reached it. The useful next formal step is to elaborate the supplied Lean migration against the pinned project and preserve the old derived `dIProg` interface. A further size claim should wait for a complete checker whose soundness, canonical completeness and entire charged source have all been established.

A Attribution for the complete inherited bridge

The following five mathematical sections are copied from `Research/certified_zero/reference/` in the pinned repository, with exact source hashes in `evidence/source_snapshot.json`. They supply the underlying logical reconstruction and arithmetic bootstrap rather than leaving the difficult nonstandard-domain steps as unstated assumptions. Any wording such as "new result" in these inherited sections refers to their original authors' investigation, not to the present direct-Q choice. The repository's MIT No Attribution license is included.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse

direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n,a;h) = I(n, I(m,a;h);h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy,a;h) = I(y,h(a);h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y,a;h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y,3a;H) = 3I(y,a;H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y,a;H)) = 3H(I(y,a;H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y,1;H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x,1;u.2u), \qquad L(x,y) = N\big(I(y,E(x);H)\big).$$

Then the following entire recursion specification is provable:

$$L(0,y) = 1, \qquad L(Sx,0) = 0, \qquad L(Sx,Sy) = L(x,y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx,Sy) &= N\big(I(Sy,2E(x);H)\big) \\ &= N\big(I(y,H(2E(x));H)\big) \\ &= N\big(I(y,3E(x);H)\big) \\ &= N\big(3I(y,E(x);H)\big) \\ &= N\big(I(y,E(x);H)\big) = L(x,y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su,m),u,Su), \qquad W(t,m) = I(t,0;u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u,v,v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b)=0$ branch, (39) gives $a=b$. In the $e(a,b)=1$ branch, an assumption $a=b$ would give $e(a,b)=0$ and therefore $0=1$, contradicting separation. Hence

$$\forall a\,\forall b\,\big(a=b\vee(a=b\to\bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t}=e(s,t), \qquad b_\bot=1, \tag{42}$$
$$b_{A\wedge D}=C(b_A,b_D,1), \tag{43}$$
$$b_{A\vee D}=C(b_A,0,b_D), \tag{44}$$
$$b_{A\to D}=C(b_A,b_D,0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A=0\vee b_A=1\big)\quad\text{and}\quad\big(b_A=0\longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A=1$ implies $\neg A$: if A held, the already established implication $A\to b_A=0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1=0\vee 1=1$, with its right disjunct proved by reflexivity. Also $1=0\to\bot$ follows from separation and symmetry, while $\bot\to 1=0$ follows by explosion. Thus $1=0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A=0\vee b_A=1$. In the zero branch, A holds and $b_{A\wedge D}=b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A\wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D}=1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A\wedge D$; conversely an assumption $A\wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D}=0$.

Disjunction. In the branch $b_A=0$, the formula A holds and $b_{A\vee D}=0$. The range is immediate, and both $b_{A\vee D}=0$ and $A\vee D$ hold, proving their equivalence. In the branch $b_A=1$, we have $\neg A$ and $b_{A\vee D}=b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A\vee D$. Conversely, from $A\vee D$, the A alternative contradicts $\neg A$ and yields $b_D=0$ by explosion, while the D alternative yields $b_D=0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G Standalone direct-closure checker

The following complete executable module is the explicit direct-Q checker measured in the main comparison. The accompanying old-rule checker uses the same shared body and the old progression fragment. The builder and proof translators are not dependencies of this module.

```
"""Standalone checker for the direct-Q native iPRA presentation.

Input is a JSON object with parameters, context, goal, and proof fields.
All syntax and scopes are checked. Only a successful run prints ACCEPT.
No certificate text is evaluated as Python. Assertions are not used.
"""
import json

SIGNATURE = {
    "z": ("term", ()),
    "s": ("term", (("term", 0),)),
    "i": ("term", (("term", 0), ("term", 0), ("term", 1))),
    "e": ("formula", (("term", 0), ("term", 0))),
    "q": ("formula", (("formula", 1),)),
    "d": ("formula", (("formula", 2),)),
}
ZERO = ["z"]

def require(condition):
    if not condition:
        raise ValueError("invalid certificate")

def variable(index):
    return ["v", index]

def validate(expression, sort, scope):
    require(type(expression) is list and bool(expression))
    tag = expression[0]
    require(type(tag) is str)
    if tag == "v":
        require(sort == "term" and len(expression) == 2)
        index = expression[1]
        require(type(index) is int and 0 <= index < scope)
        return
    require(tag in SIGNATURE)
    expected_sort, children = SIGNATURE[tag]
    require(sort == expected_sort and len(expression) == len(children) + 1)
    for child, (child_sort, bound) in zip(expression[1:], children):
        validate(child, child_sort, scope + bound)

def substitute(expression, images):
    if expression[0] == "v":
        return images(expression[1])
    children = SIGNATURE[expression[0]][1]
    return [expression[0]] + [
        substitute(child, lift(images, bound))
        for child, (_, bound) in zip(expression[1:], children)
    ]

def lift(images, bound):
    return lambda index: (variable(index) if index < bound
                          else shift(images(index - bound), bound))
```

```
def shift(expression, amount=1):
    return substitute(expression, lambda index: variable(index + amount))

def instantiate(expression, term):
    return substitute(expression, lambda index: term if index == 0
                      else variable(index - 1))

def successor_instance(expression):
    return substitute(expression, lambda index: ["s", variable(0)]
                      if index == 0 else variable(index))

def witness_instance(matrix, witness):
    return substitute(matrix, lambda index: variable(0) if index == 0
                      else shift(witness) if index == 1
                      else variable(index - 1))

def derive(proof, context, scope):
    require(type(proof) is list and bool(proof))
    rule, *arguments = proof
    shifted_context = [shift(formula) for formula in context]
    if rule == "hyp":
        index, = arguments
        require(type(index) is int and 0 <= index < len(context))
        return context[index]
    if rule == "zero":
        seed, update = arguments
        validate(seed, "term", scope)
        validate(update, "term", scope + 1)
        return ["e", ["i", ZERO, seed, update], seed]
    if rule == "succ":
        counter, seed, update = arguments
        validate(counter, "term", scope)
        validate(seed, "term", scope)
        validate(update, "term", scope + 1)
        source = ["i", ["s", counter], seed, update]
        result = instantiate(update, ["i", counter, seed, update])
        return ["e", source, result]
    if rule == "tr":
        template, equality_proof, instance_proof = arguments
        validate(template, "formula", scope + 1)
        equality = derive(equality_proof, context, scope)
        require(equality[0] == "e")
        require(derive(instance_proof, context, scope)
                == instantiate(template, equality[1]))
        return instantiate(template, equality[2])
    if rule == "qe":
        term, closure_proof, base_proof = arguments
        validate(term, "term", scope)
        closure = derive(closure_proof, context, scope)
        require(closure[0] == "q")
        require(derive(base_proof, context, scope)
                == instantiate(closure[1], ZERO))
        return instantiate(closure[1], term)
    if rule == "de":
        conclusion, major_proof, branch_proof = arguments
        validate(conclusion, "formula", scope)
        major = derive(major_proof, context, scope)
        require(major[0] == "d")
        branch_context = [["q", major[1]]] + shifted_context
        require(derive(branch_proof, branch_context, scope + 1)
```

```
                == shift(conclusion))
        return conclusion
    if rule == "case":
        matrix, witness, branch_proof = arguments
        validate(matrix, "formula", scope + 2)
        validate(witness, "term", scope)
        body = witness_instance(matrix, witness)
        branch_context = [shift(instantiate(body, ZERO))] + shifted_context
        require(derive(branch_proof, branch_context, scope + 1)
                == successor_instance(body))
        return ["d", matrix]
    if rule == "prog":
        equation, branch_proof = arguments
        validate(equation, "formula", scope + 1)
        require(equation[0] == "e")
        branch_context = [equation] + shifted_context
        require(derive(branch_proof, branch_context, scope + 1)
                == successor_instance(equation))
        return ["q", equation]
    raise ValueError("unknown rule")

def check(document):
    scope = document["parameters"]
    context = document["context"]
    goal = document["goal"]
    require(type(scope) is int and scope >= 0 and type(context) is list)
    for formula in context:
        validate(formula, "formula", scope)
    validate(goal, "formula", scope)
    require(derive(document["proof"], context, scope) == goal)

if __name__ == "__main__":
    check(json.load(open(0)))
    print("ACCEPT")
```

References

[1] Vladimir Reshetnikov, repository maintainer. *Concise presentations of intuitionistic primitive recursive arithmetic. ProveIt*, pinned commit `b17dd5487f34adea473a0204c5320fb3eeb8910e`. https://github.com/VladimirReshetnikov/ProveIt/tree/b17dd5487f34adea473a0204c5320fb3eeb8910e/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] *ConciseIPRA/NativeEq.lean*, especially `NPrf.qIProg` and `NPrf.eqInd`, in [1]. https://github.com/VladimirReshetnikov/ProveIt/blob/b17dd5487f34adea473a0204c5320fb3eeb8910e/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeEq.lean.

[3] *ConciseIPRA/BiInterpretation.lean*, in [1]. https://github.com/VladimirReshetnikov/ProveIt/blob/b17dd5487f34adea473a0204c5320fb3eeb8910e/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/BiInterpretation.lean.

[4] *ConciseIPRA/NativeComposite.lean*, in [1]. https://github.com/VladimirReshetnikov/ProveIt/blob/b17dd5487f34adea473a0204c5320fb3eeb8910e/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeComposite.lean.

[5] *Research/certified_zero/reference/*, in [1]: the inherited Choice-DI basis, ordinary bridge, scalar predecessor, equality reflection and primitive-recursion compiler. Reproduced above with source provenance.

[6] *Research/neutral_scope*, *Research/prevalidated_elimination*, and *Research/coverage_and_kernel*, in [1]. Exact selected checker modules and hashes are distributed in `reference/checkers/` and `evidence/source_snapshot.json`.

[7] Inherited scalar predecessor development, reproduced as `reference/scalar_predecessor.tex` in [5]. The original fragment's attribution is preserved.

[8] Inherited forward-left equality-reflection development, reproduced as `reference/equality_reflection.tex` in [5].

[9] Inherited pairing and primitive-recursion compiler, reproduced as `reference/primitive_recursion.tex` in [5].